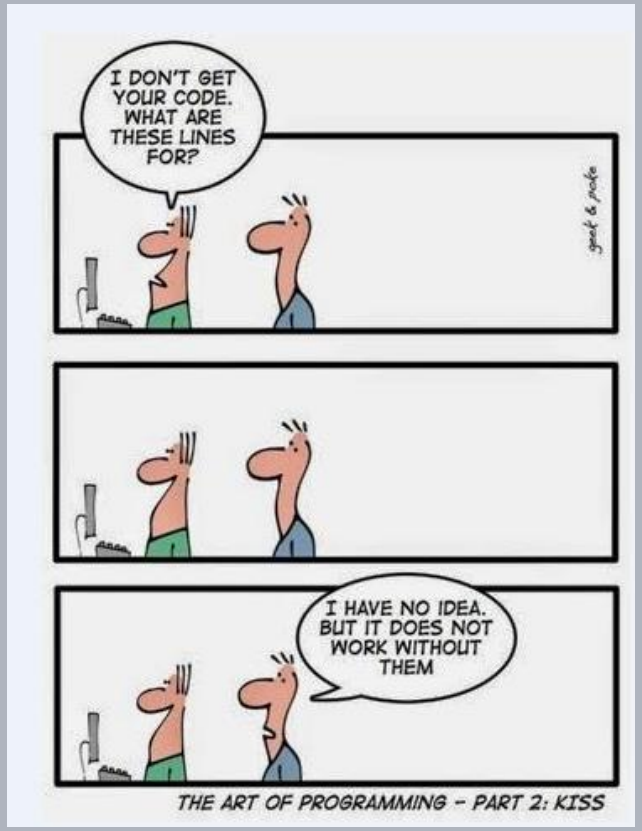
Advanced Software Design Techniques

Elementary Structural Design Patterns

Prof. Dr.-Ing. Peter Fromm



Content

Design goals
- Cohesion and Coupling
- Stability versus changeability

Elementary methods
- OOA and OOD
- Decomposition (partitions and hierarchies)
- Generalisation and Specialisation
- Configurations and Variants

More advanced methods
- Model View Controller
- Input Logic Output
- Single Responsibility Method
- Liskov Substitution Principle
- Dependency Inversion Principle
- Hardware Abstraction

A high level view on a system

The architecture of a system is the shape given to the system by those who built it. The form of that shape is the division of that system into components, the arrangement of those components, and the way in which those components communicate with each other.

Robert C. Martin, Clean Architecture

Development	Deployment	Operation	Maintenance
The Architecture should make it easy for the team to develop the system.	The Architecture should support an easy and cheap deployment – preferably one single action.	The system should run on the smallest possible hardware.	Maintenance is the most expensive parts. Changes must be easy and robust.

Cohesion and Coupling

Cohesion

In computer programming, cohesion refers **to the degree to which the elements inside a module belong together**. In one sense, it is a measure of the strength of relationship between the methods and data of a class and some unifying purpose or concept served by that class. In another sense, it is a measure of the strength of relationship between the class's methods and data themselves.

Cohesion is an ordinal type of measurement and is usually described as “high cohesion” or “low cohesion”. Modules with high cohesion tend to be preferable, because high cohesion is associated with several desirable traits of software including robustness, reliability, reusability, and understandability. In contrast, low cohesion is associated with undesirable traits such as being difficult to maintain, test, reuse, or even understand.

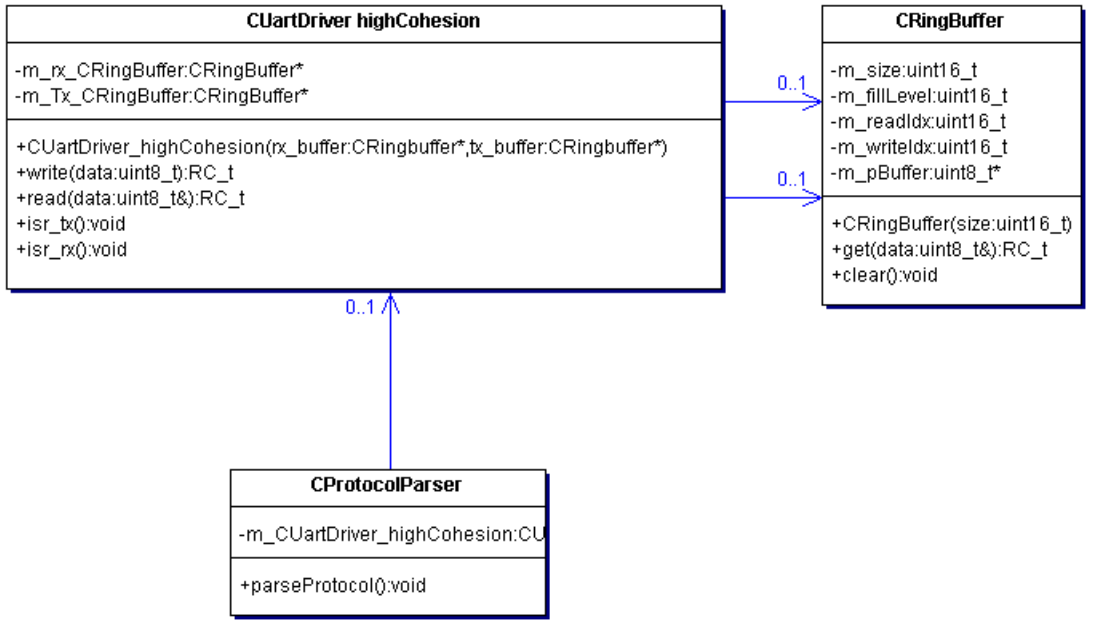
Source: Wikipedia

Example high versus low cohesion

- If you can clearly describe the responsibility of a component, class or function in **one single meaningful sentence**, the cohesion very likely is high.

Example UART_driver:

- The job of the driver is to read / write data to a UART peripheral and to store it in a buffer.



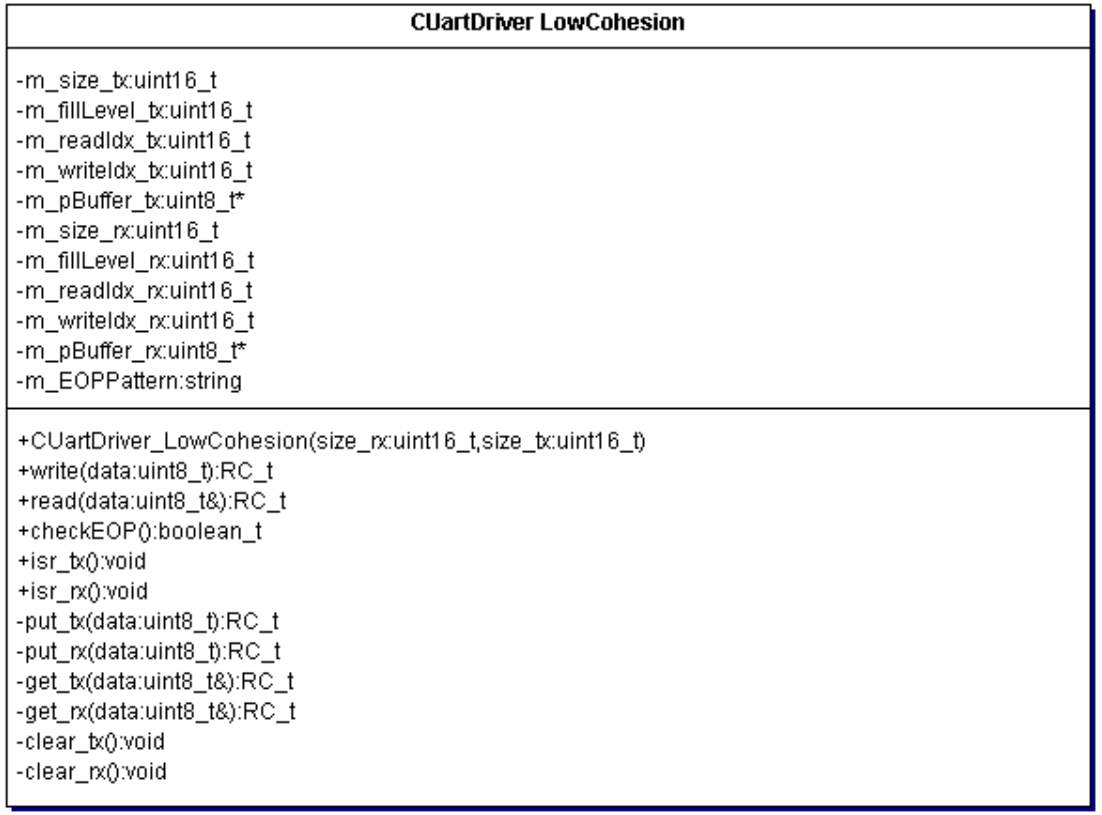
Example high versus low cohesion

Example UART_driver:

- The job of the driver is to read / write data to a UART peripheral using a ringbuffer for buffering transmission and reception data.
- The two buffers are part of the class.
- Furthermore the driver checks for a complete protocol by analysing the postamble of the incoming data stream.

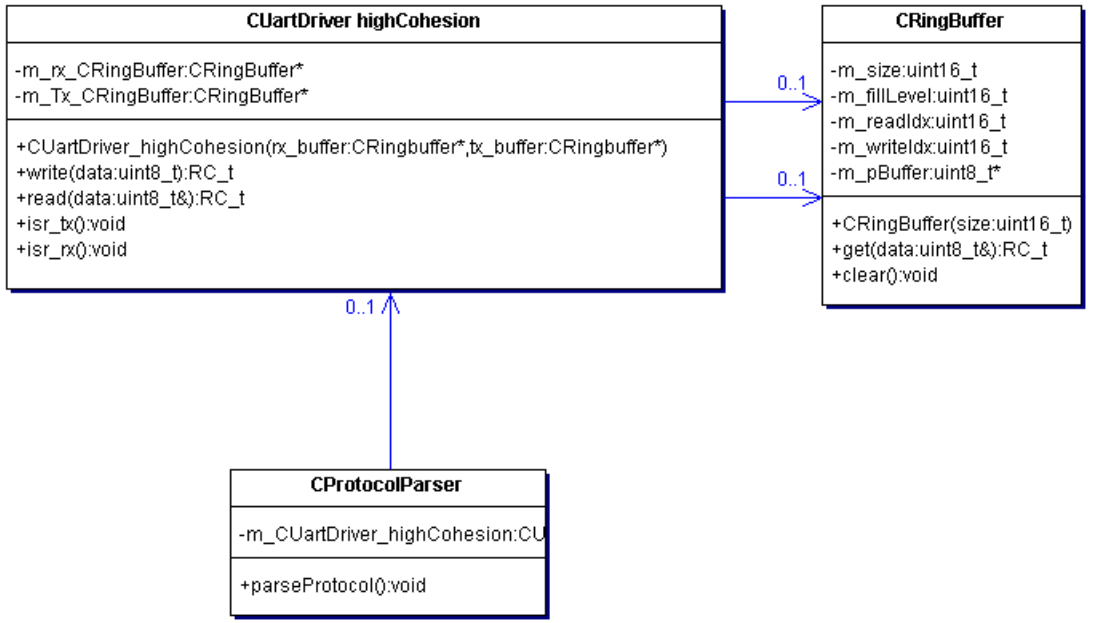
Problem: Three different stories are mixed up, making the design clumsy, hard to use and inflexible and creating large test efforts

- UART driver
- Buffer
- Protocol definition



Advantage of good cohesion: Easier and more flexible re-use

- Moving to a new hardware: `CUartDriver` needs to be replaced
- Extending the Protocol: `CProtocolParser` needs to be replaced
- `CProtocolParser` can be used for both communication platforms, independent of hardware, ensuring protocol consistency.
- `CRingBuffer`: You always need it!



Cohesion and Coupling

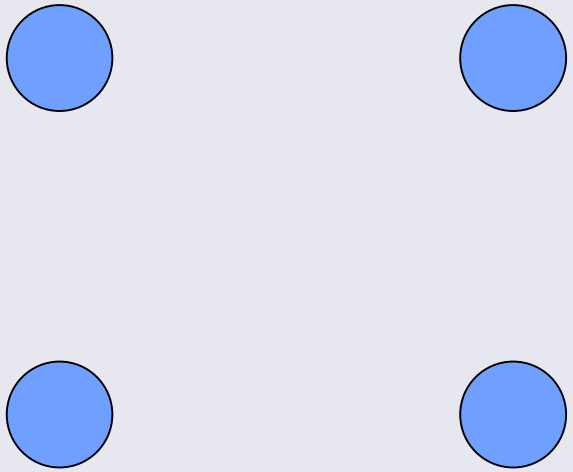
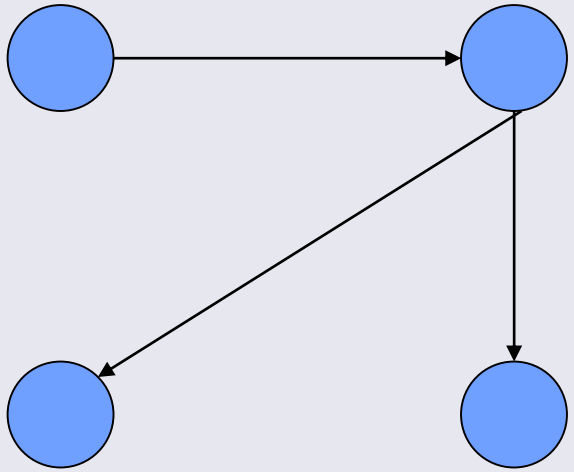
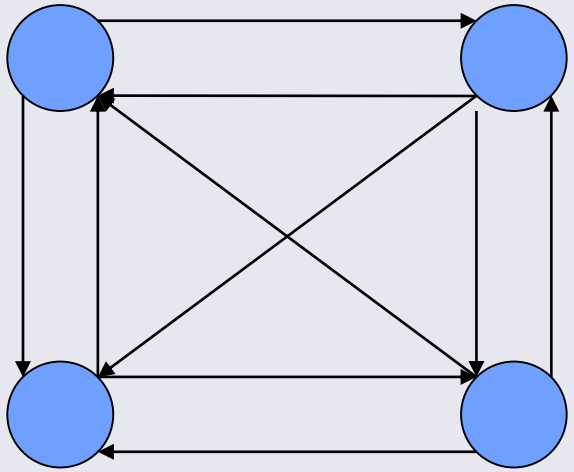
Coupling

In software engineering, coupling is the degree of interdependence between software modules; a measure of how closely connected two routines or modules are; the strength of the relationships between modules.

Coupling is usually contrasted with cohesion. Low coupling often correlates with high cohesion, and vice versa. Low coupling is often a sign of a well-structured computer system and a good design, and when combined with high cohesion, supports the general goals of high readability and maintainability.

Source: Wikipedia

Coupling

Uncoupled	Loosely coupled	Highly coupled
Components act independently, no cooperation. Clear, individual responsibilities.	Components act together, but responsibilities are still visible	Many dependencies, high risk for side effects after changes, ambiguous responsibilities.
		

Type of Coupling

Tight coupling

- Subclass coupling (aggregation, composition, association, inheritance)
- Function coupling (calling a function of another module)
- Common Data coupling (global data)

Loose coupling

- Type coupling (UML dependency)
- Dynamic coupling (polymorphism, function pointers, callbacks)
- Data coupling (data exchange by parameters, through a well defined API, e.g. blackboard)

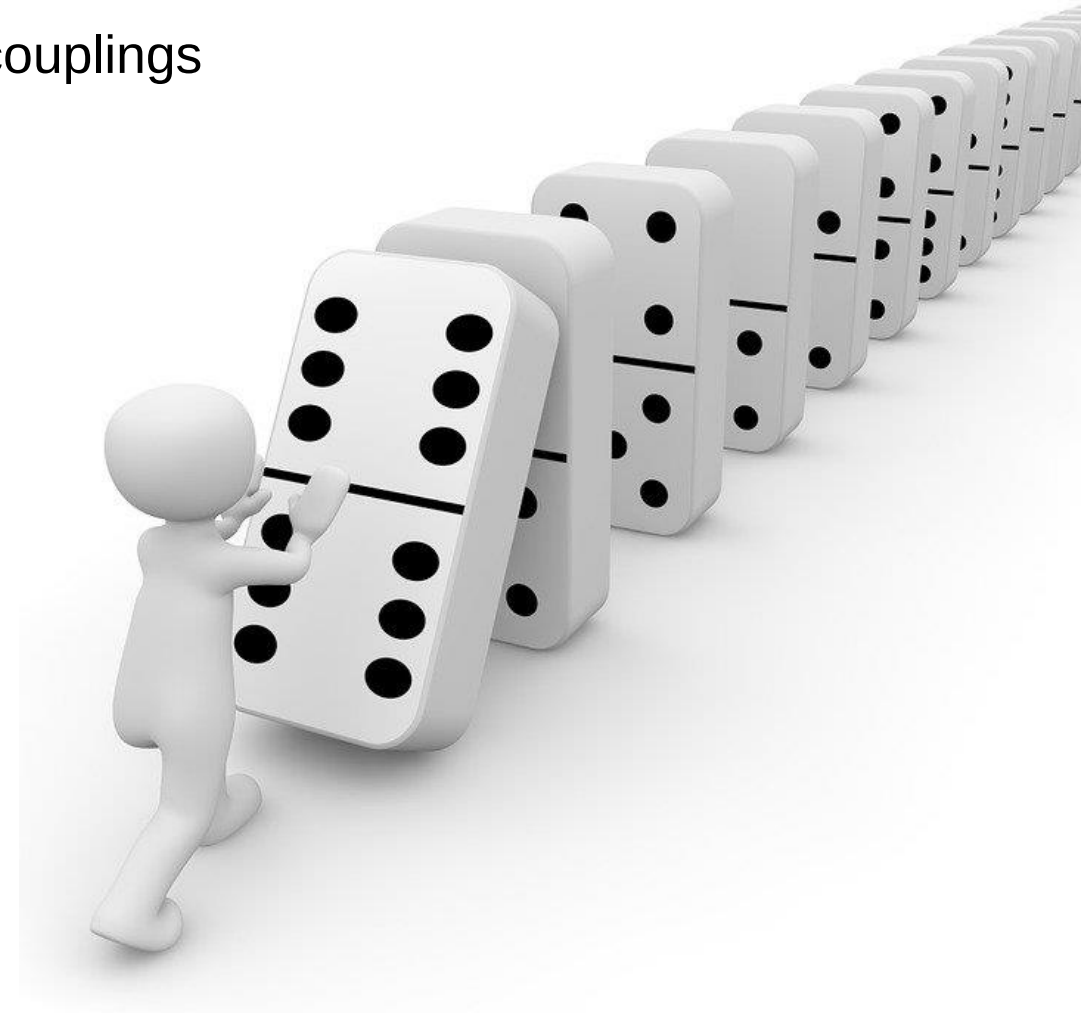
Problems

- Circular dependencies (A needs B, B needs C, C needs A)
  - Redesign (separating the critical types in own files)
  - Using forward declarations

#include “…”

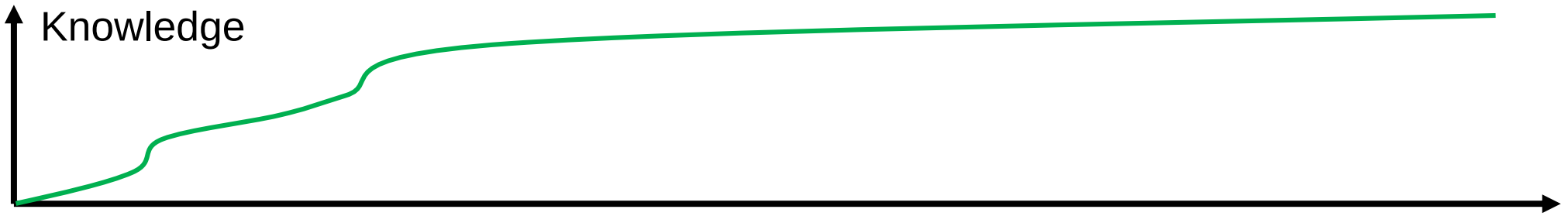
Houston, we have a problem

- To reduce coupling, programmers tend move code into single components
- This results in low cohesion
- Increasing the risks of even more un-intended couplings
- …



Key Design Goals:
1) Construct small classes of high cohesion
2) Create unidirectional dependencies – not the number is the problem, but a chaotic structure!
3) Use loose coupling

Remember this curve?



Try to get as much knowledge as early as possible: Prototyping, skinny sheep, modelling, customer interaction,…

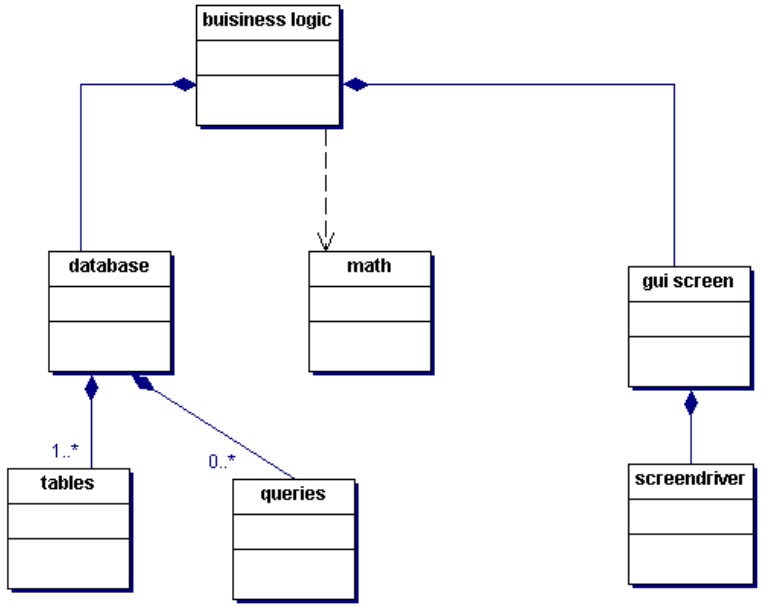
Try to remain flexible – move decisions to the end. But be able to implement them in a save way during this late phase!

Stability versus changeability

- Which parts of the design are easily changeable (i.e. no side effects expected), which parts must remain stable (i.e. should not be modified).

Rules

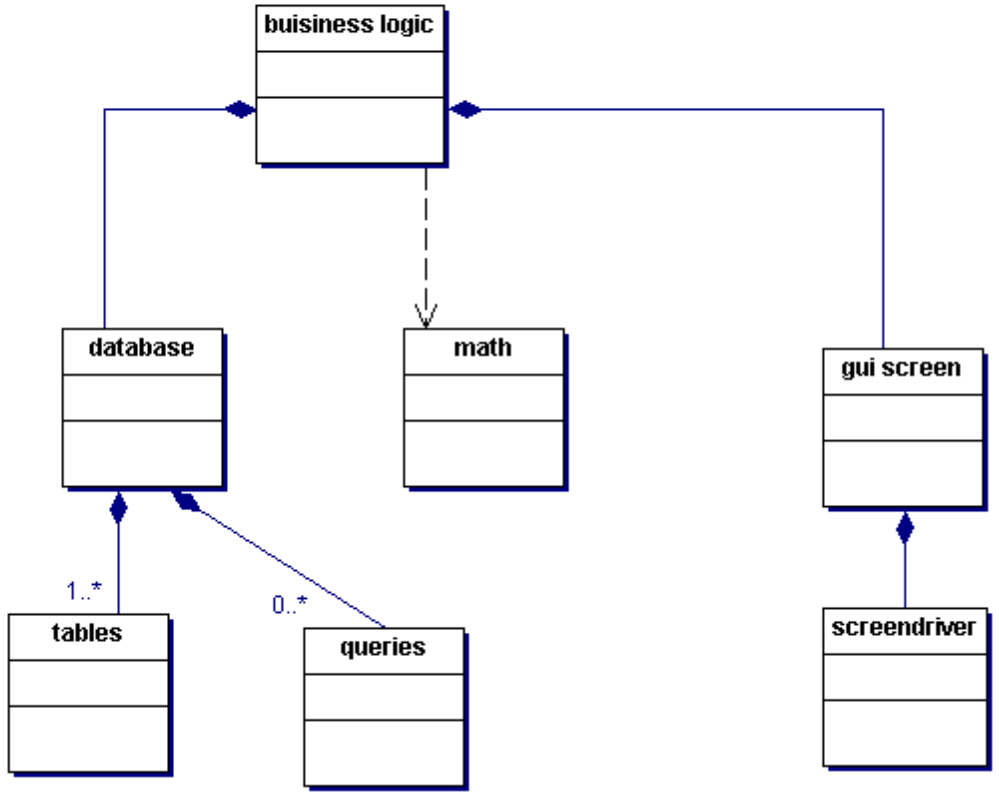
- Small and generic classes with high cohesion tend to be stable.
- Classes implementing the customer visible behaviour will (likely) be changed.
- Classes depending on volatile technology will be (need to be) replaced
- Classes low in the hierarchy should be stable



Where do we expect changes?

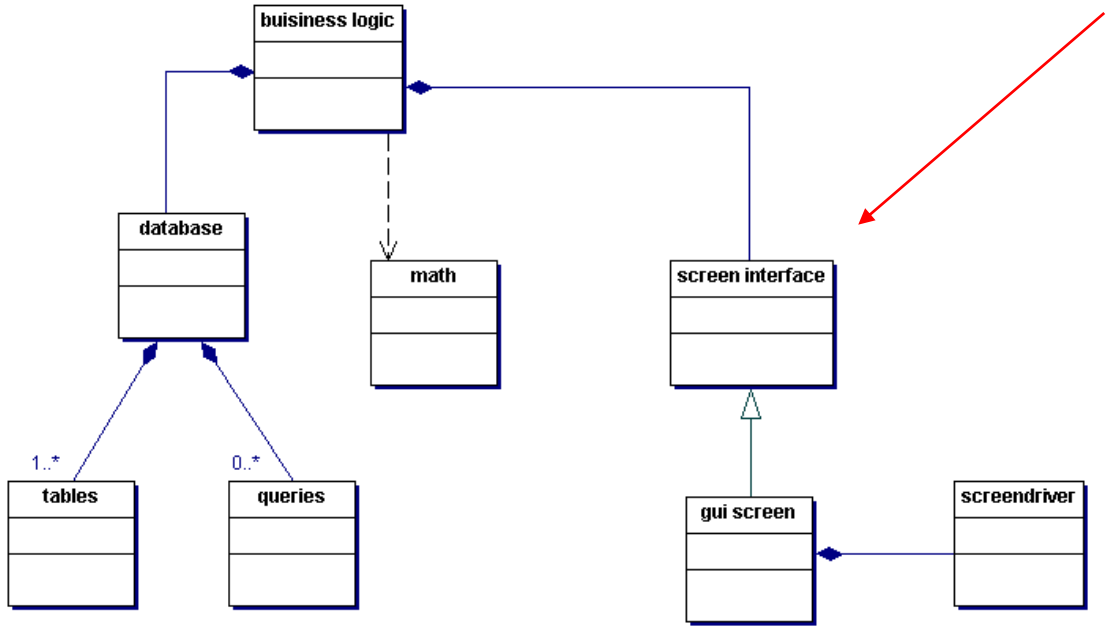
Let’s compare different solutions – version 1

- Changes in the business logic (driven by user needs) will have an impact on the database and screen – in-avoidable.
- Changes in the GUI screen class may affect the business logic – unwanted.
- A change in screen technology may affect the rest of the system. Not very likely, as such driver tend to be stable. At least from the API perspective.
- Math class can be considered to be rather stable.



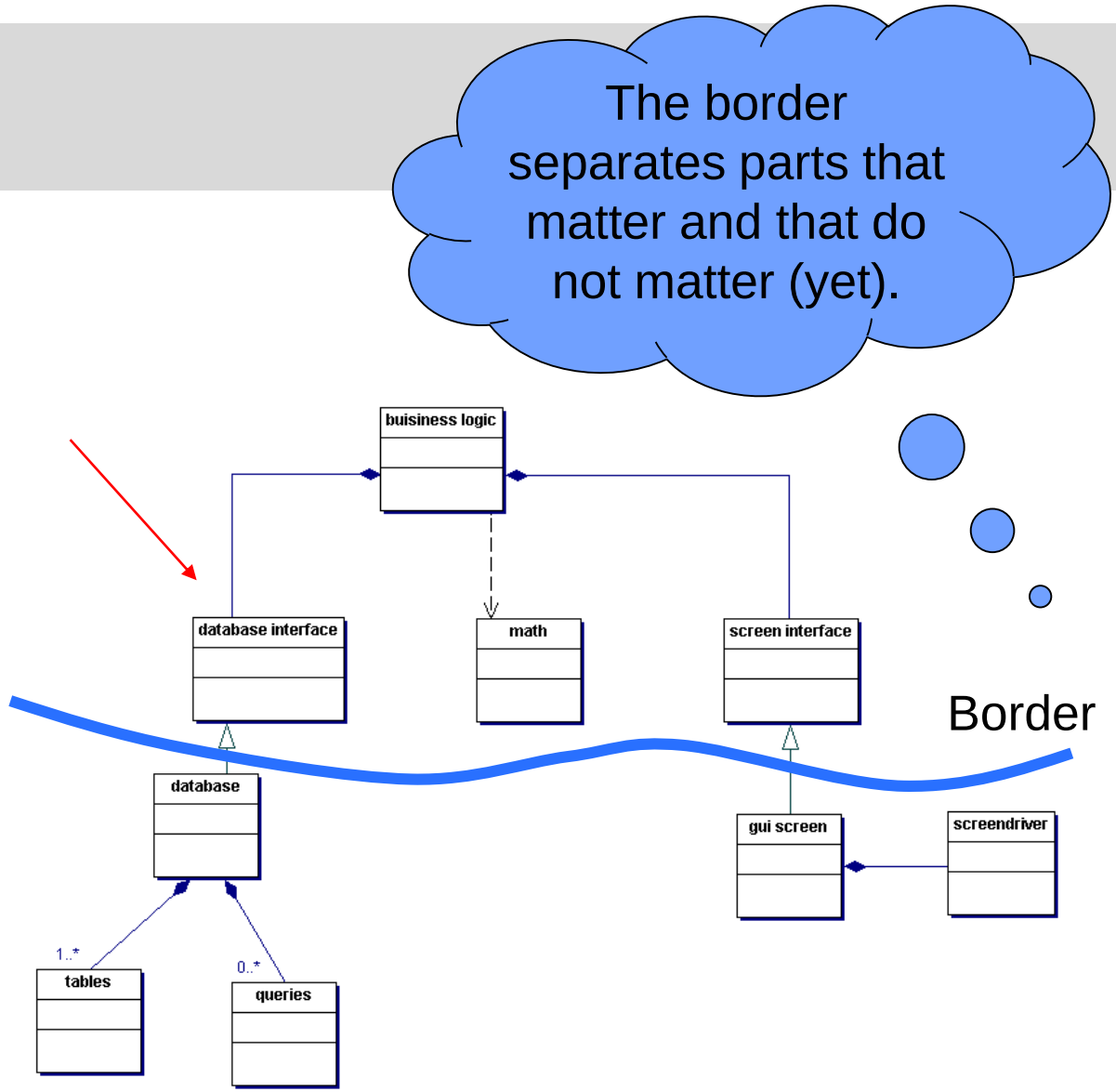
Let’s compare different solutions – version 2

- The screen layout very likely will be changed by the user.
- Therefore it makes sense to add an interface which is inverting the dependency.
- As long as the concrete screen implementation complies with the agreed interface, it can be changed as needed.



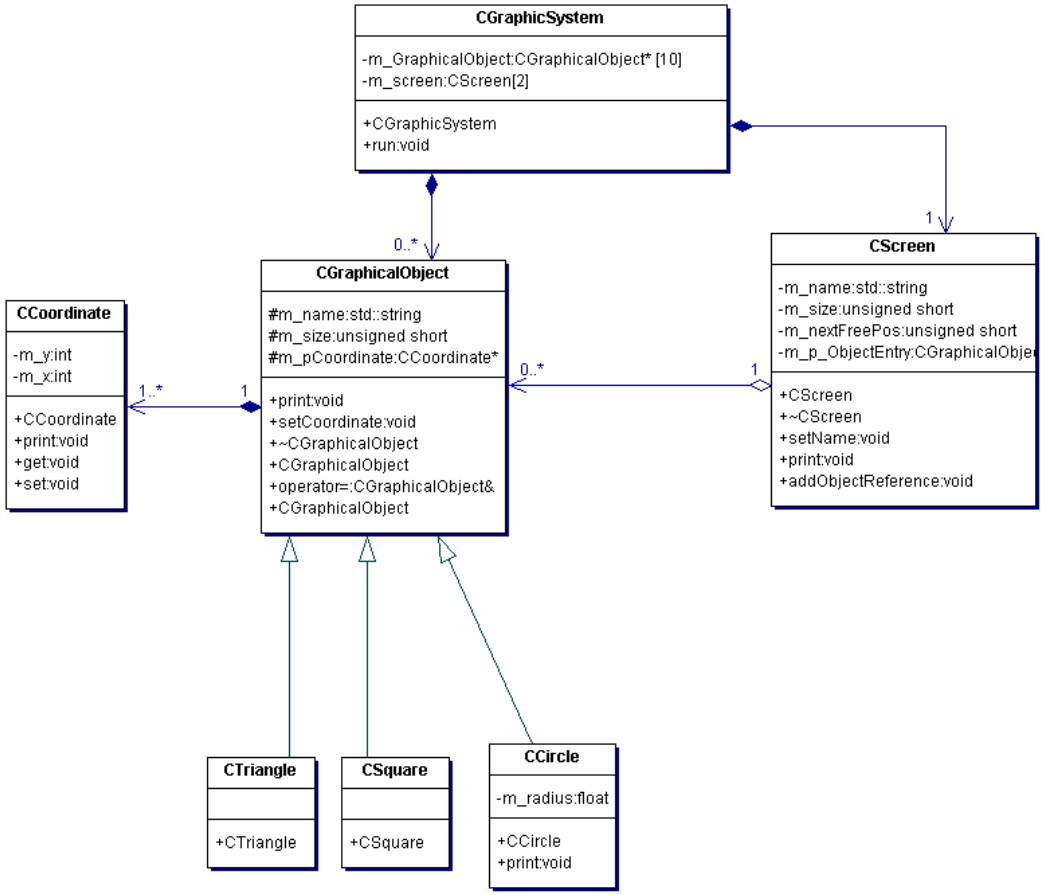
Let’s compare different solutions – version 3

- Adding an interface class for the database will allow us to easily change database technology, e.g. replace a local SQL database with a web service or similar.
- This will not save us from changing the database structure based on business logic modifications.
- But in a first implementation we can focus on implementing the business logic, trying to get this as stable as possible to reduce late changes from this side.
- Downside: Designing a generic database API is challenging!



The OO-approach

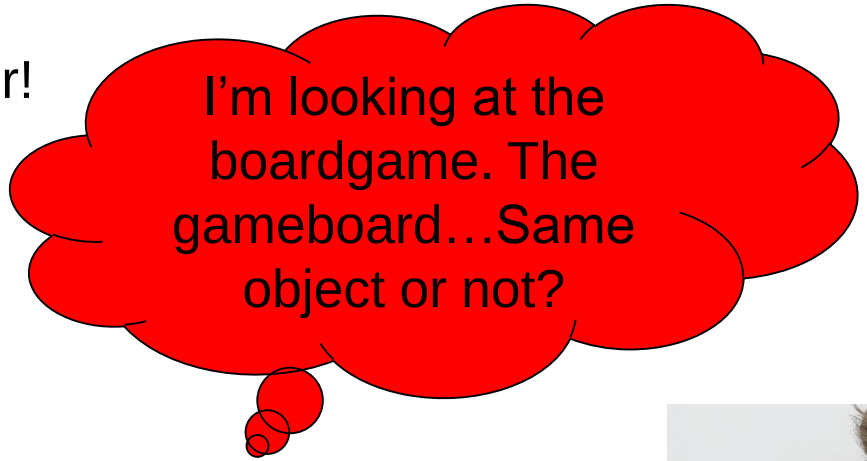
- Object Oriented Analysis is a very powerful method to identify first components and classes, by describing the “real world” or by looking at the nouns of a requirement specification.
- Classes are small building blocks, facilitating reuse.
- Class relations can be identified by using the phrases
  - Is-part-of ➔ composition or aggregation
  - Contains ➔ composition or aggregation
  - Knows ➔ association
  - Is-a ➔ inheritance
- Attributes and Methods can be identified by describing the properties and capabilities of a class.

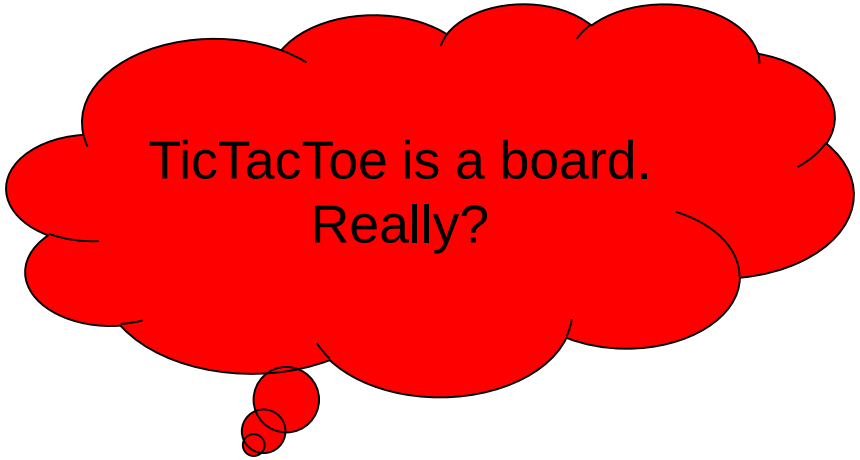


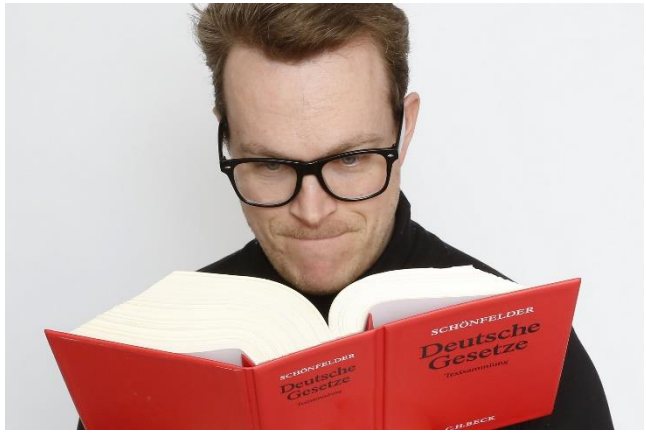
The OO-approach
Good Naming

The starting point for an OO analysis: “look at the nouns of the specification”…

- Challenge: we have to find good words!
- Example TicTacToe
- Be picky!!!! Pretend you are a lawyer!







Partitions and Hierarchies

A core technology to structure software is decomposition. The software is broken into smaller parts, until a component level is reached, which can be implemented by a developer or small team.

Partition

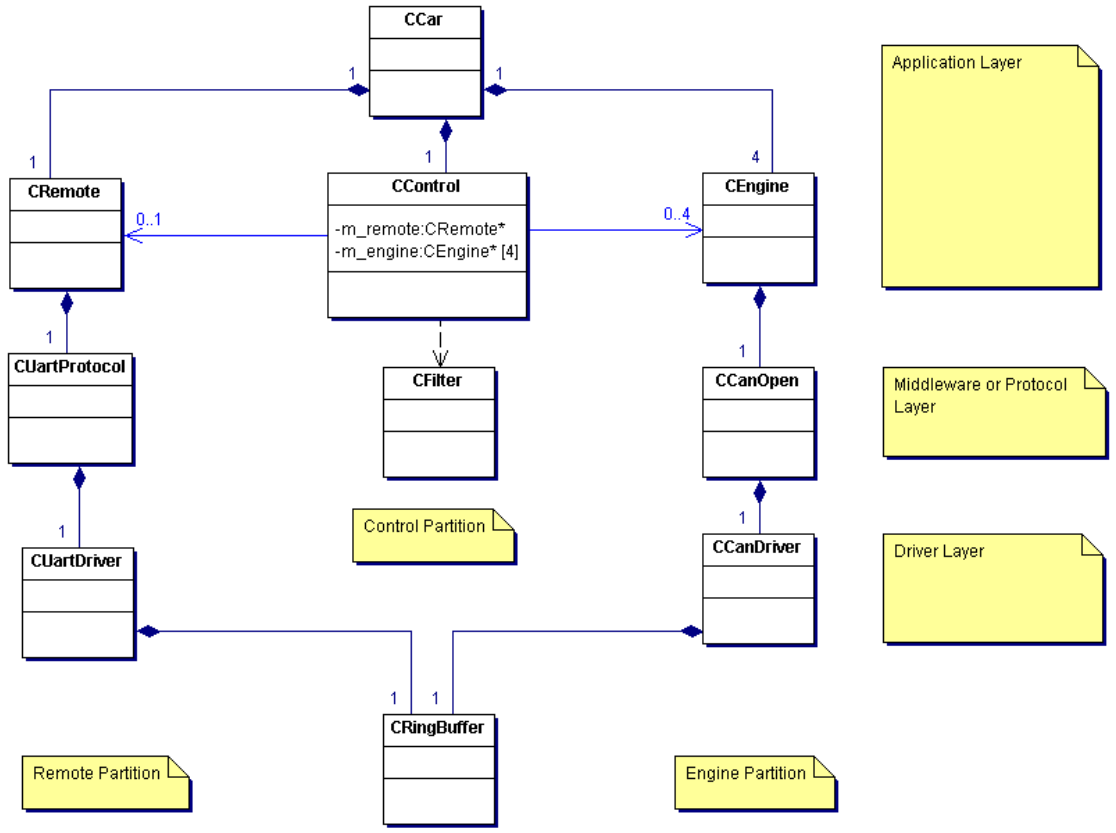
- Partitioning divides the software into independent parts. Partitions may be implemented in parallel. Example: UART Driver and PWM Driver

Hierarchy

- In a hierarchy, one software component depends on another one. Using this approach, the lower components should be implemented first. Example: hardware driver, protocol stack, application

Partitions and Hierarchies

- Only very few dependencies between partitions (only application level)
- Unidirectional dependencies in Remote and Engine Partition
- Clear responsibilities per hierarchical level
- Bidirectional and circular dependencies are avoided!
- Obvious order of building and testing the classes.



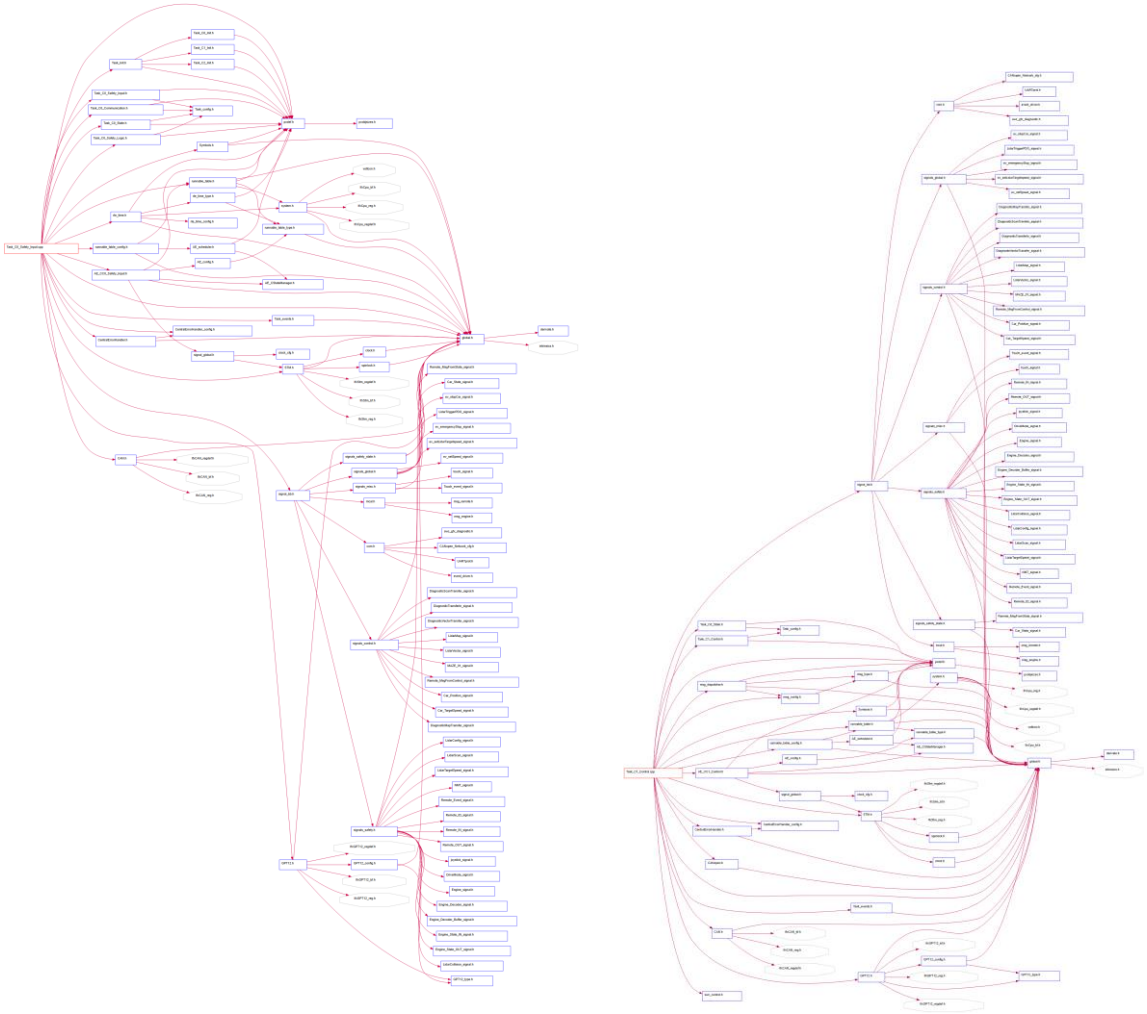
Reality looks more complex…

Include graph of

- Safety task
- Application Task

of the StudentCar

- Partitions: not so many dependencies vertically.
- Hierarchy: All partitions are interfaced to signal layer



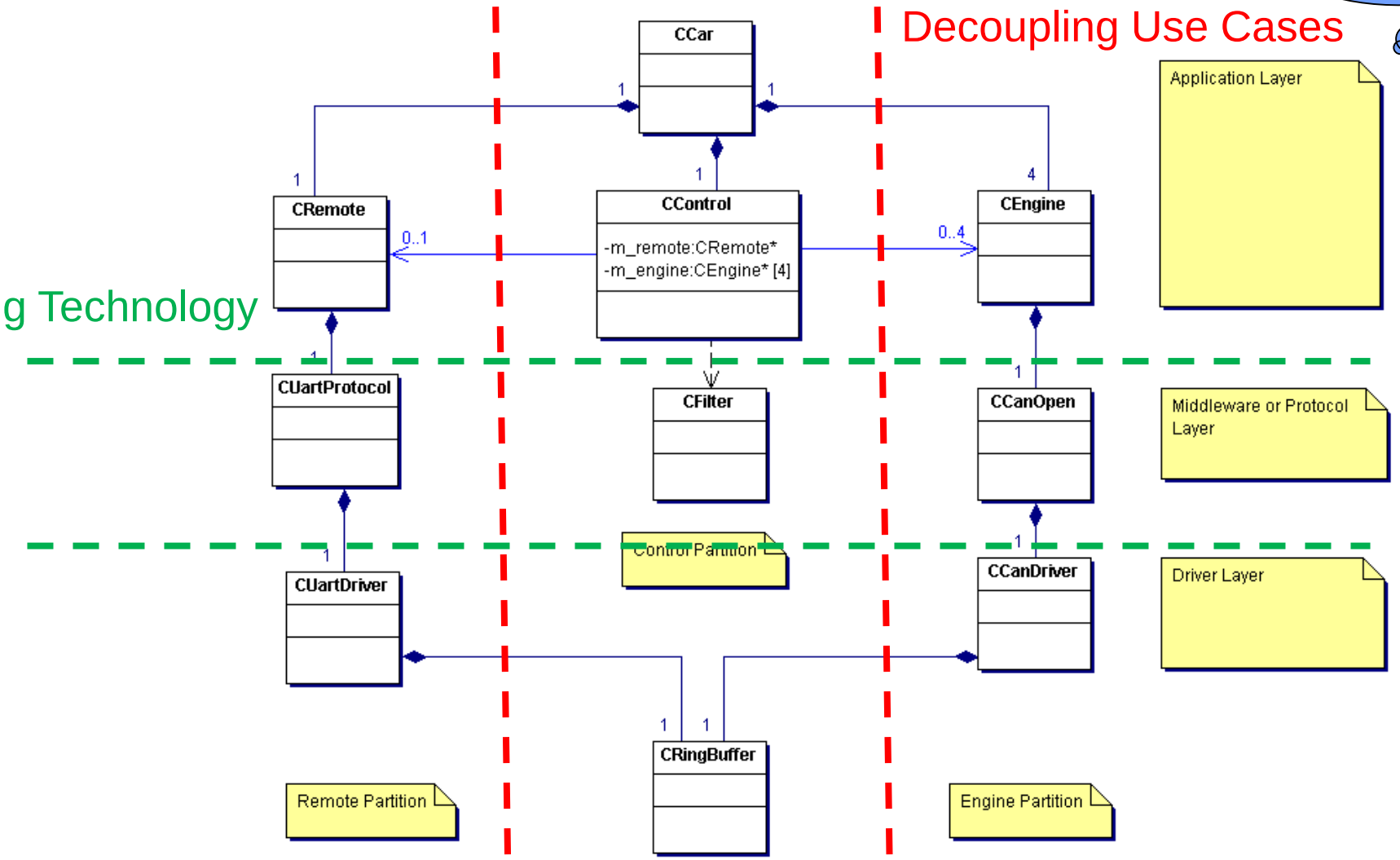
Limitation of the include graph: we don't know what kind of coupling we have. And not all are shown (data, function pointers)…

Decoupling through Layers and Partitions

Translates into (partly) independent workpackages

Decoupling Use Cases

Decoupling Technology



Design for Testability

Interesting enough, a good structure of layers and partitions also supports the proncpile of design for testability.

Problem:

- When changing a system, testcases which have been implemented before very often do not work anymore.
- Example: The structure of a login screen has changed and the test robot does not know where to place the data.
- This leads to a rigid design, as changes become expensive.

Solution:

- Develop testcases working on the stable parts of the design, e.g. not on the system level GUI, but directly on the business logic classes
- You still will need some extra testcases for the GUI, but way less!

Generalisation and Specialisation

Changing code is hard, extending code is easy!

Change:

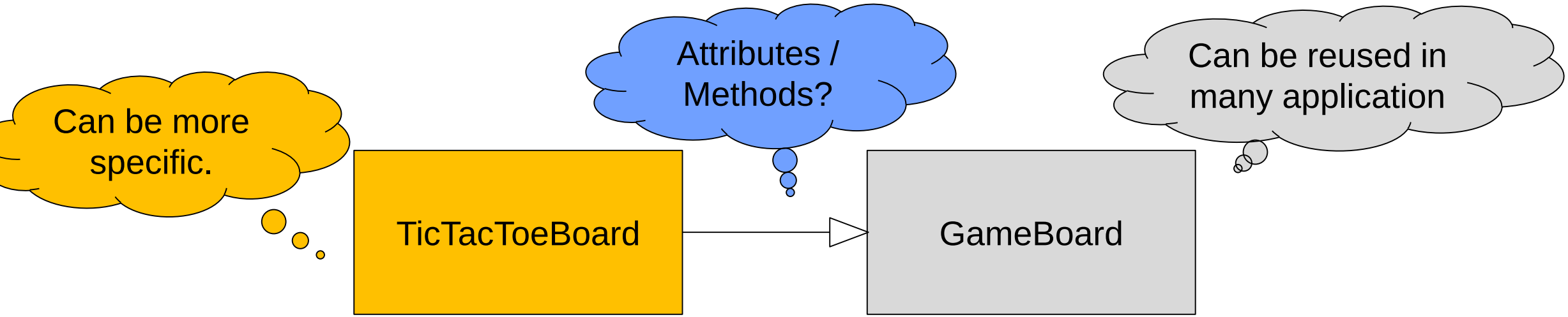
- Risk of side effects through coupling
- Domino effect: Several classes have to be modified as a consequence of the initial change
- Complete functionality must be retested
- Changes behind the interface are less critical

Extension

- Existing code is not modified, less risk of side effects (although not zero)
- Less impact on coupled classes
- More obvious regression test strategy: Only added code needs to be qualified

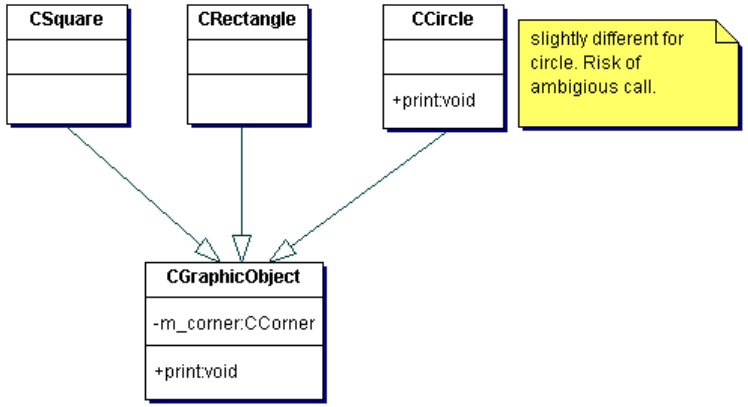
Generalisation and Specialisation

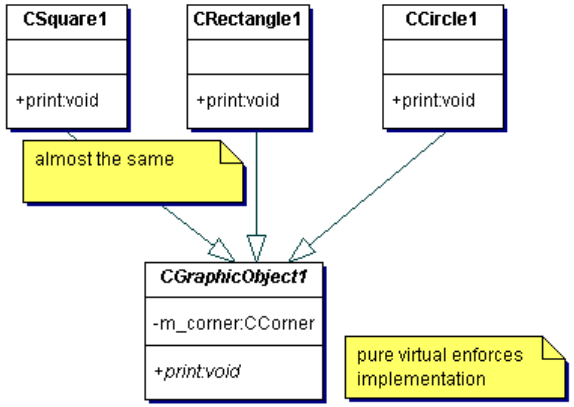
- Classes which should be used in many projects need to be generic and stable (often containing general services like math functions, I/O operations, data structures,…)
- Classes which will be used in a single project can be more specific (often containing the use case business logic)
- Use inheritance to separate the specific part from the generic part

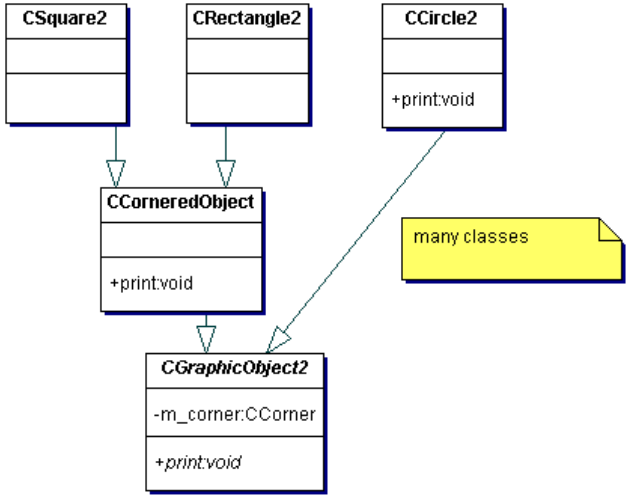


Generalisation and Specialisation using Inheritance

There is no silver bullet…







Some more advanced stuff

- Model View Controller
- Input Logic Output
- Single responsibility rule
- Load Balancing
- Liskov Substitution Principle
- Dependency Inversion Principle
- Hardware Abstraction

Model View Controller

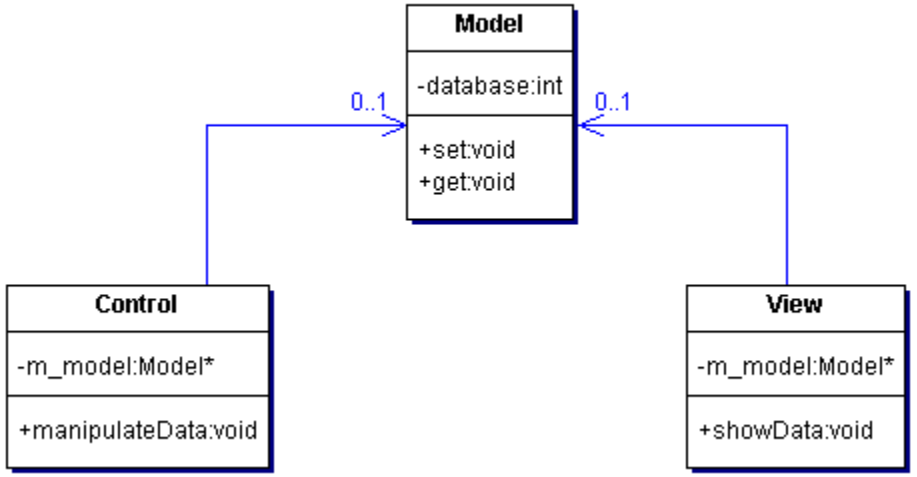
The Model View Controller Pattern separates programs, involving user interactions, into three interconnected elements.

Advantages

- Flexibility, especially concerning the view

Disadvantages

- Data centric - modifications in the model will affect all other components



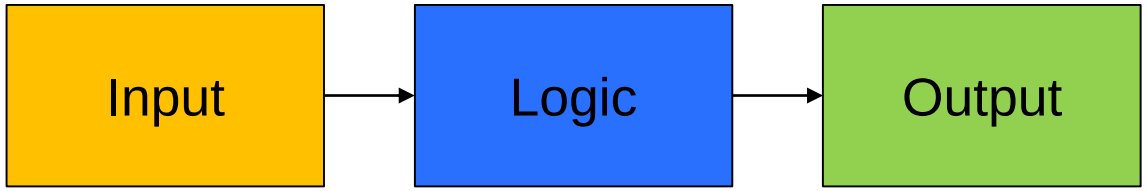
Input Logic Output

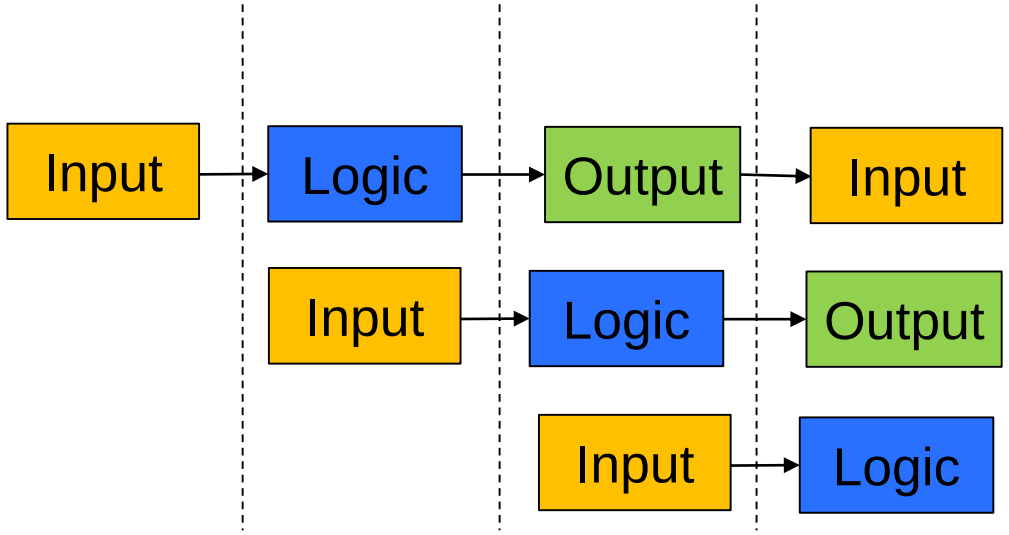
Mainly used for control applications. The focus is a clear temporal consistency. In every cycle, the sensor data first is refreshed, then the actor values are calculated, followed by the actors being set. The temporal separation usually requires also a structural separation.

Advantages
- Temporal consistency

Disadvantages
- Risk of long latencies
  (e.g due to slow sensors)

Possible Solution
- Multithreading
- Might require complex synchronisation

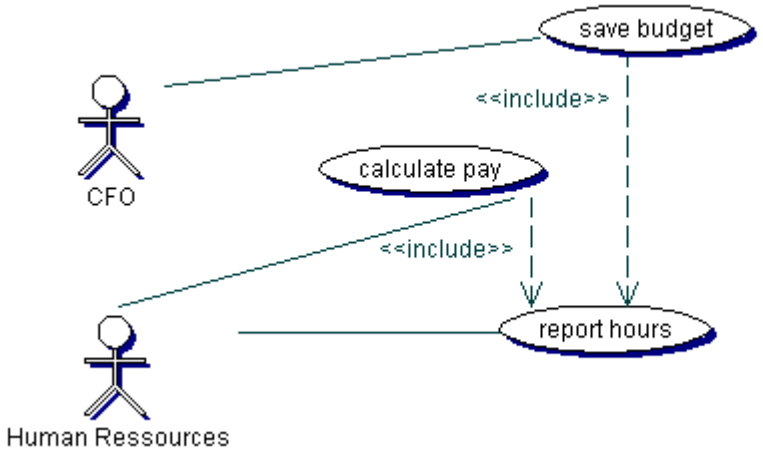


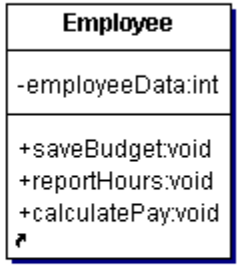


Single Responsibility Rule

The single responsibility rule comes in different flavours, all of equal importance

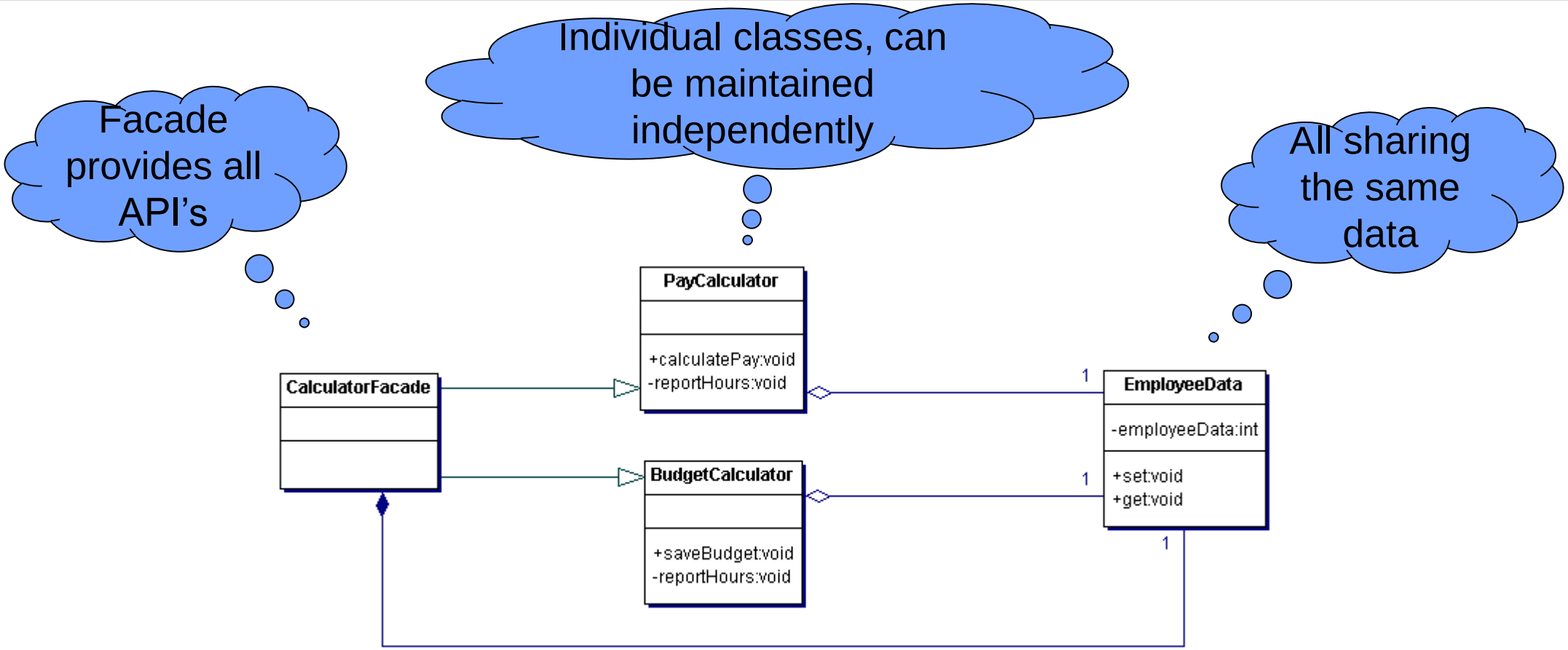
- A class should have one job / responsibility – to avoid confusion, which class has to be used for a specific task.
- A class should have only one reason / stakeholder for modifications – to avoid contradicting requirements and chaotic changes.



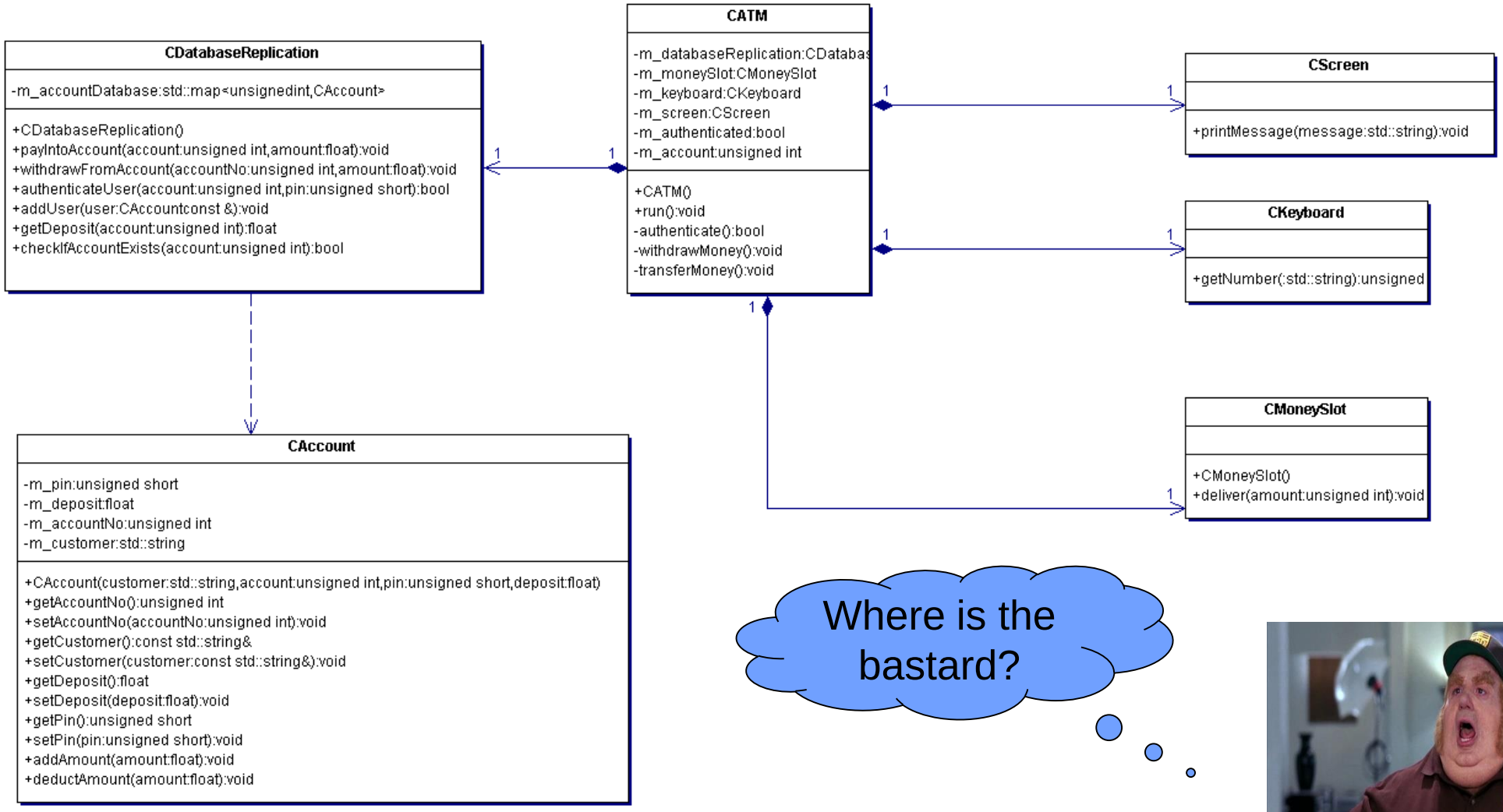


Problem: The algorithm for calculating the hours might be different!

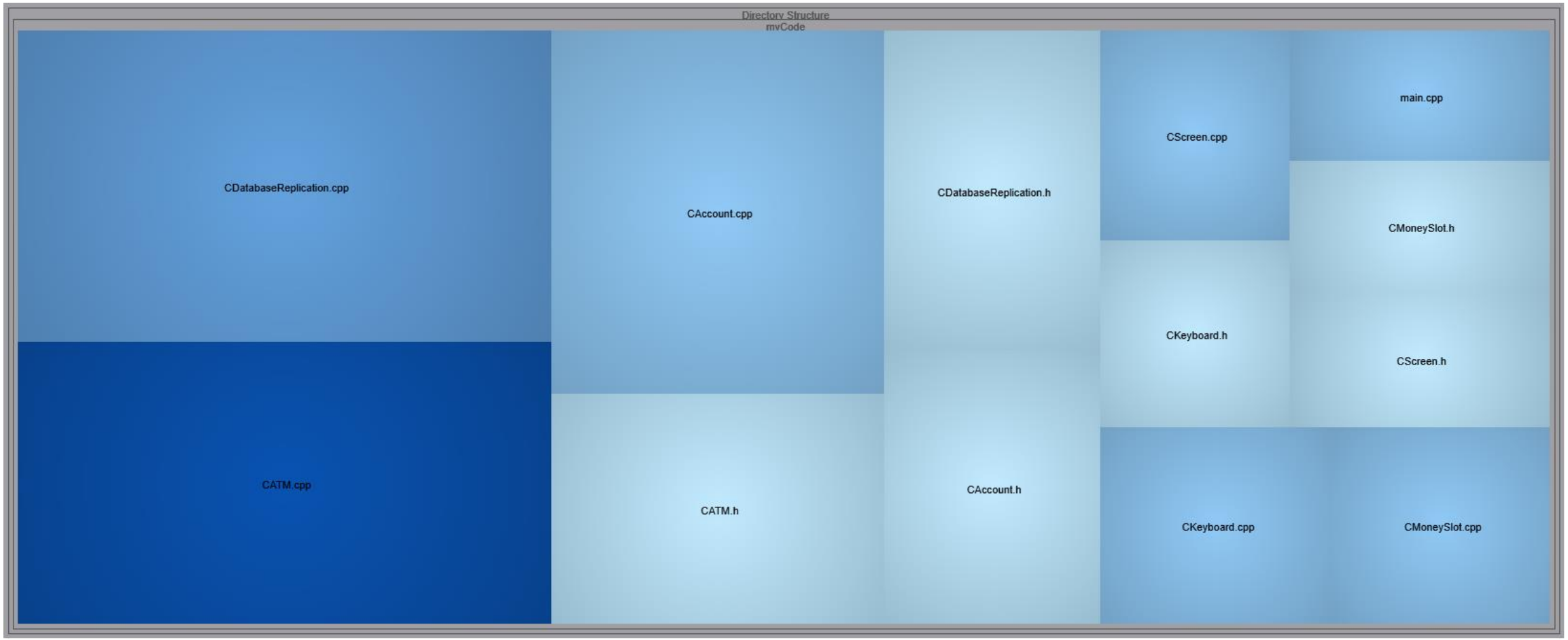
Single Responsibility Rule



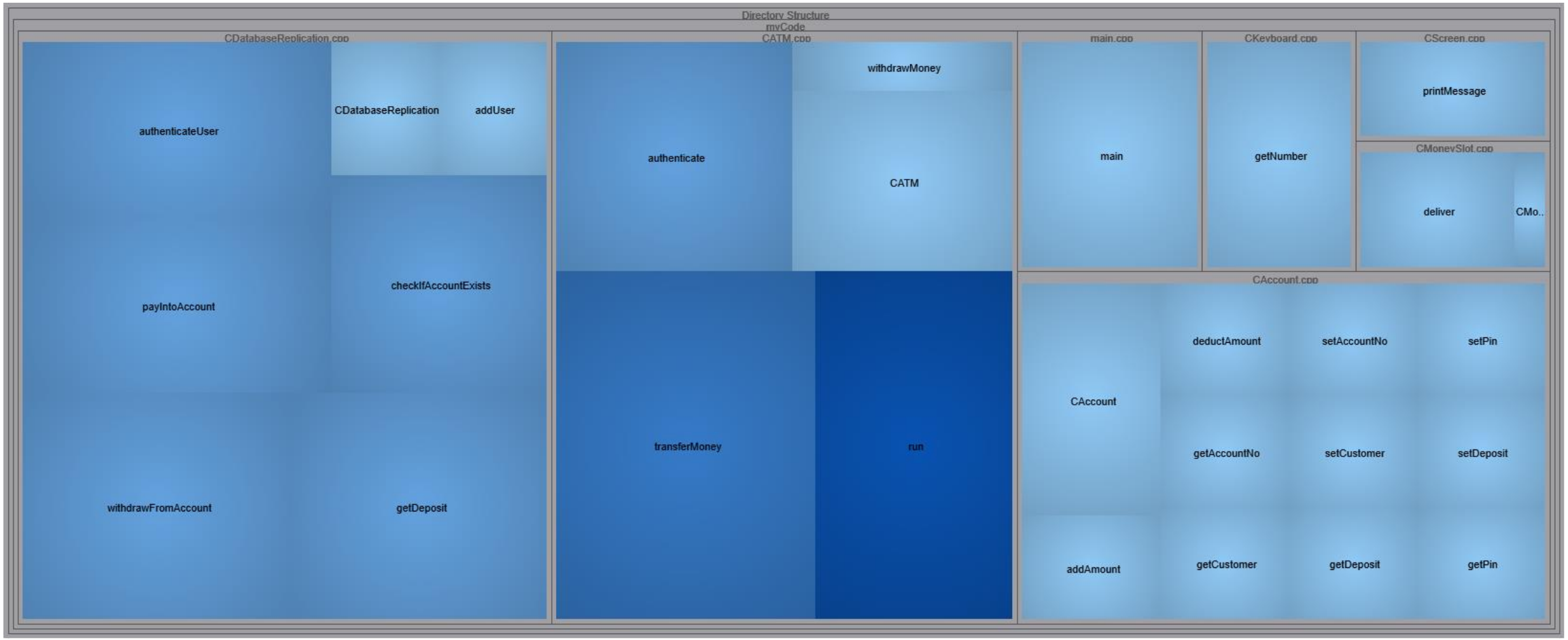
Load Balancing



Load Balancing – File Level (size versus complexity)



Load Balancing – Function Level (size versus complexity)



Load Balancing

Typical problem in star designs:

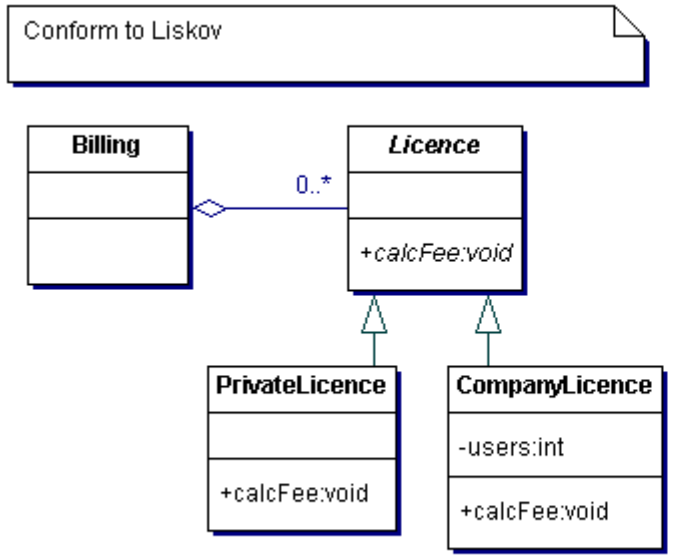
- Many small classes profile simple functions
- Main class collects all the data and does all the (complex) logic

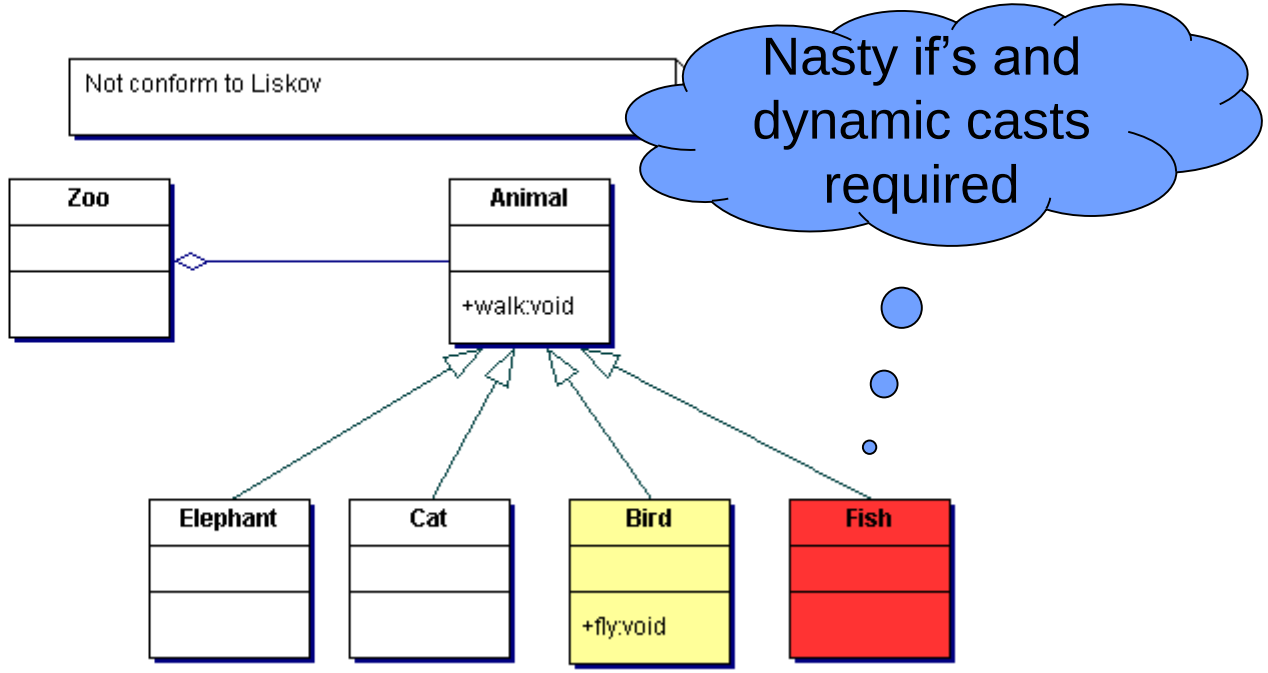
Try to find services – example Database class!

Liskov Substitution Principle

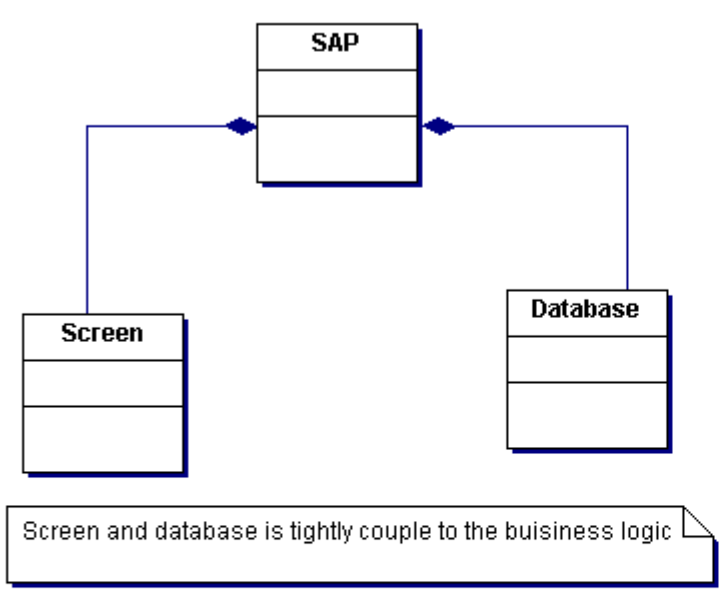
In 1988, Barbara Liskov made the following recommendation for defining subtypes:

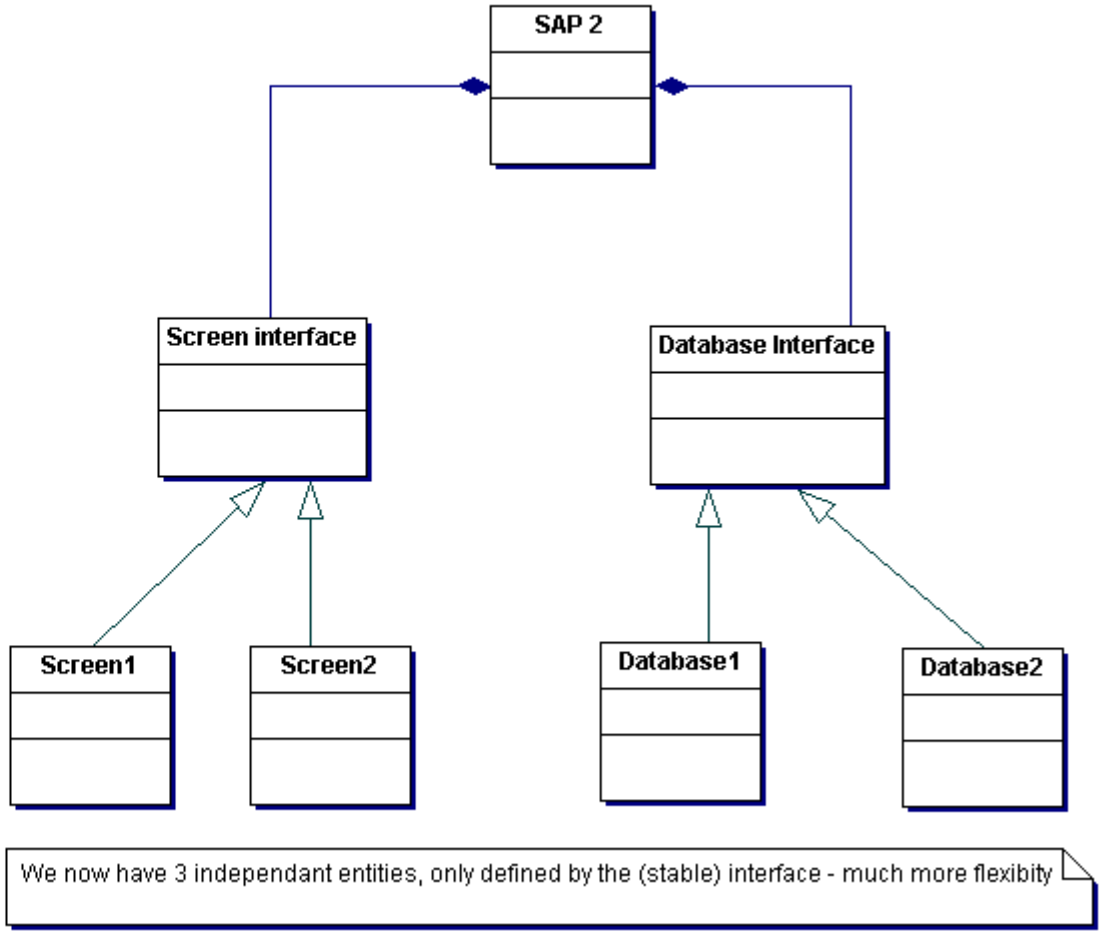
What is wanted here is something like the following substitution property: If for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behaviour of P is unchanged when o1 is substituted for o2, then S is a subtype of T.





Dependency Inversion Principle



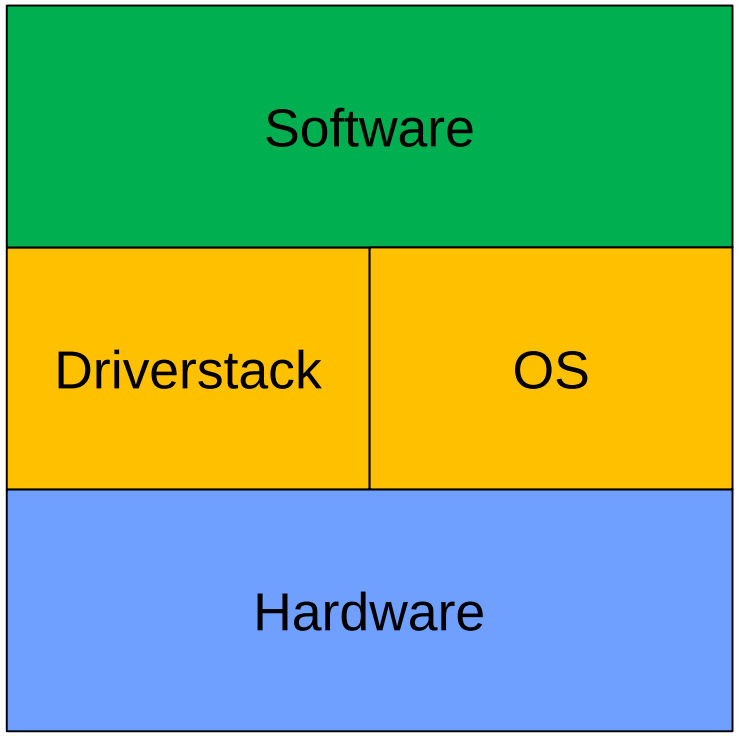


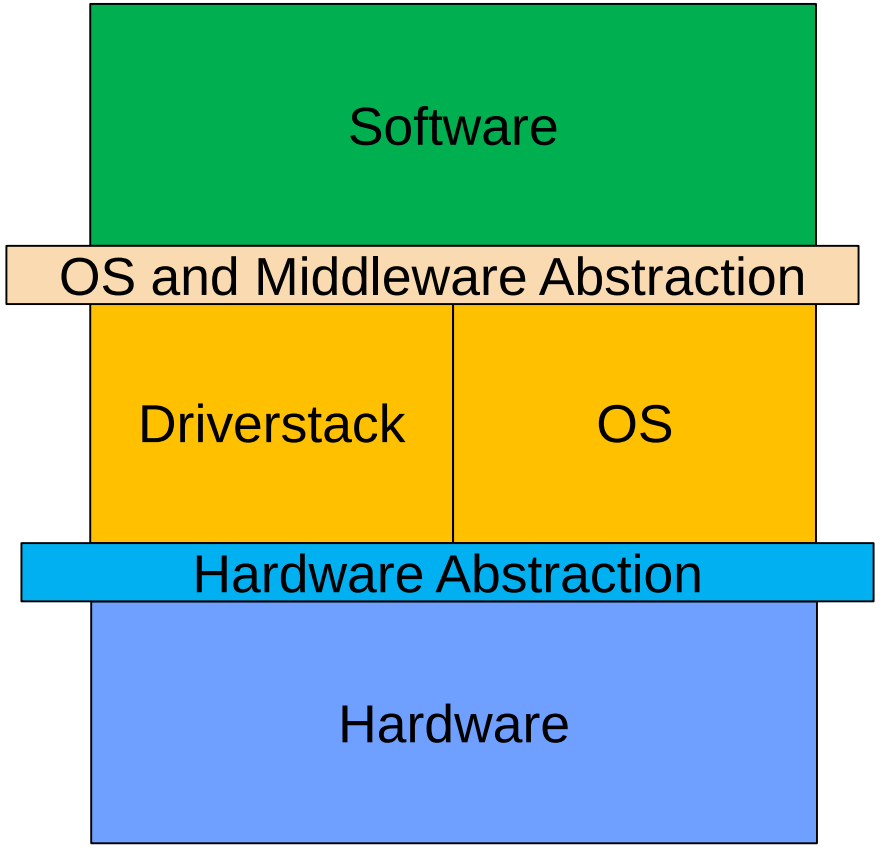
Embedded Systems

In embedded systems, most of the concepts apply as well (of course), but there might be some constraints:

- No C++ support. OK, we could use void* function pointers instead of polymorphism, but readability might suffer
- Need for optimisation. True, but we do not have 8bit controller anymore. At least not so often. Most resources are wasted due to bad design, not due to bad coding!

Hardware abstraction





Example of “abstract” embedded code

Machine independent data types

However the driver checks it..

We don’t care how we get the data

Now we work on the data

Calculate the target speed

Again, we don’t care about the driver

```
void CONTROL0_run_manual(void){

sint8_t loc_x = JOYSTICK_NULL;
sint8_t loc_y = JOYSTICK_NULL;
sint8_t loc_Rz = JOYSTICK_NULL;

signalStatus_t joystick_sig_stat;
joystick_data_t joystick_data;
TargetSpeed_data_t target_speed;

joystick_sig_stat=sd_joystick.getStatus();
joystick_data=sd_joystick.get();

if (SIG_STS_VALID == joystick_sig_stat)
{
          loc_x = joystick_data.x_in;
          loc_y = joystick_data.y_in;
          loc_Rz = joystick_data.Rz_in;

}

target_speed.vx = loc_x;
target_speed.vy = loc_y;
target_speed.vphi = loc_Rz;

sd_Car_TargetSpeed.set(target_speed);

}
```

Key Philosophy not only for Embedded Developers

1. Make it work. *You will be out of business if it doesn't work*
2. Then make it right. *Refactor the code so that you and others can undrrstand it and evolve it as needs change or are better understood*
3. Then make it fast. *Refactor the code for "needed" performance*

Robert C. Martin – Clean Architecture

Wrap-Up

- Design High-Cohesion and Low (uni-directional) Coupling
- Create small units having clear responsibilities
- Be aware of hierarchical order versus partitions
- Separate reuse code from specific code
- Think about configuration options